Predictive Modeling for Energy Level and Stability Determination of Organic Photovoltaics

Report Template



CREATING MEANINGFUL EXPERIENCES

Academy of Data Science & AI

Block 1B Machine Learning 2025-26

Breda University OF APPLIED SCIENCES

Contents

1 Business Understanding	**2**
2 Data Understanding	**2**
3 Data Preparation	**2**
4 Classification	**2**
5 Regression	**3**
6 Clustering	**3**
7 Limitations	**3**
8 Conclusion	**3**
References	**4**

Business Understanding

The global transition toward renewable energy made organic photovoltaics, or ''OPVs'', a promising alternative to traditional silicon-based solar cells. Unlike their unorganic opponents, OPVs offer unique advantages including flexibility, lightweight construction, tuneable optical properties, and the potential for low-cost and large-scale production. However, the field faces a significant challenge: the vast chemical selection of potential organic materials makes experimental testing of every candidate molecule way too expensive and time-consuming.

The primary stakeholders in this project include materials scientists, solar cell manufacturers, and research institutions focused on renewable energy development. But it could be used in a vast range of use cases in the future. These stakeholders face a critical bottleneck in the discovery process—identifying which molecular structures has the optimal combination of energy levels (HOMO and LUMO) and long-term stability before committing resources to synthesis and testing. Traditional trial-and-error approaches can take months or years to identify promising candidates, which consumes investor's budget, and market opportunities may be missed.

Machine learning offers a solution to this challenge by enabling prediction of key molecular properties from chemical structure alone. By training predictive models on existing experimental data, we can rapidly screen thousands of candidate molecules, identifying the most promising structures for synthesis and testing. This approach dramatically accelerates the discovery pipeline while reducing costs

This project aims to develop robust machine learning models capable of predicting three critical properties of molecules directly from their molecular structure.

First one on the list is Stability Classification: Predicting whether a molecule will exhibit high, medium, or low stability. Stability is crucial for commercial usage, as OPV devices must maintain performance over extended periods of exposure to light and heat.

Second, HOMO Energy Prediction: Accurately estimating the energy of the highest occupied molecular orbital, which governs the molecule's electron-donating capability and directly influences the open-circuit voltage of resulting solar cells.

LUMO Energy Prediction: Predicting the energy of the lowest unoccupied molecular orbital, which determines the molecule's electron-accepting properties and affects charge separation efficiency.

Several challenges may arise from the nature of this problem, first there are the data limitations, we face the classic machine learning challenge of limited number of samples relative to the high-dimensional chemical descriptor space. This creates a fair risk of overfitting where models might learn patterns specific only for the given training set

and therefore do not generalize well. Second, orbital energies exhibit non-linear dependencies which may be a lot more difficult for traditional machine learning algorithms to capture correctly.

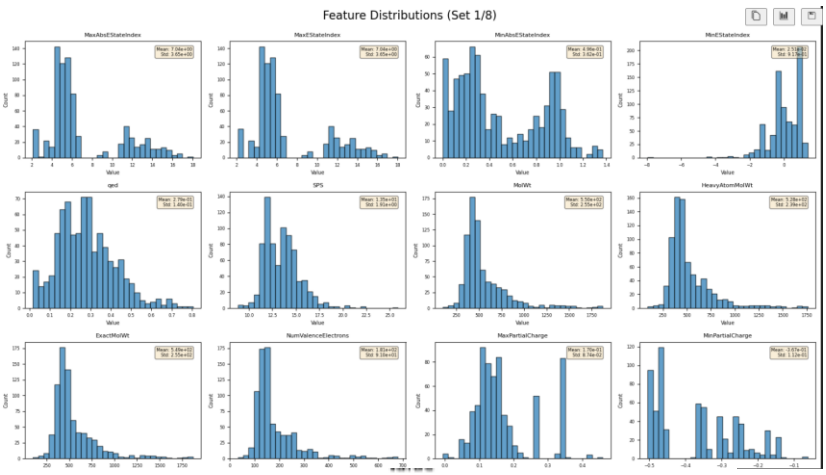
Data Understanding

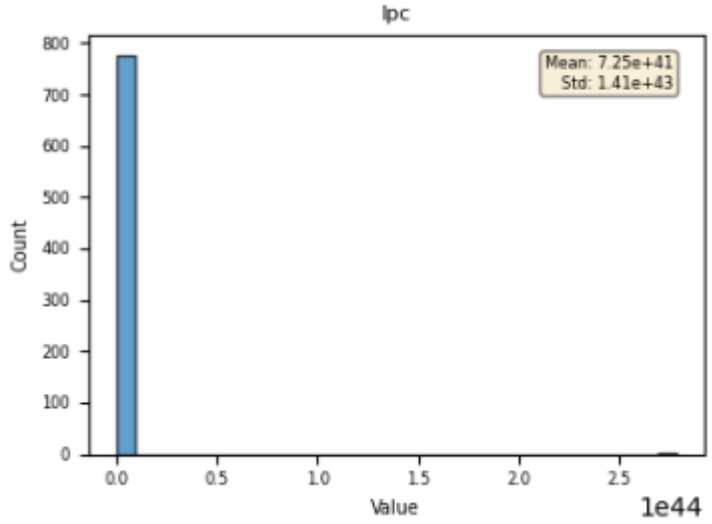
At first, I worked with 350 molecules form Harward Organic Photovoltaic Dataset, which I got from two sources, one was HOPV_stability.csv, a straightforward CSV file containing 350 SMILES strings paired with stability classifications (high, medium, or low). This file was directly readable as a pandas DataFrame, requiring minimal preprocessing beyond duplicate removal. Second was HOPV_homolumo.data, a complex text file containing almost 375,000 lines of data for the same 350 molecules. Unlike the CSV format, this file required sophisticated parsing to extract HOMO and LUMO values along with additional experimental metadata. The file structure follows a specific pattern where each molecule's data spans approximately 1,072 lines, with the SMILES string appearing on line 0, an InChI identifier on line 1, experimental measurements on line 2, a repeated SMILES on line 3, and unused data on lines below until it started with a smile string again . We developed a custom parser function to navigate this structure, extracting SMILES strings and experimental values while discarding the rest of the data.

The initial check revealed some quality issues, three smiles strings appeared as duplicates in the stability dataset, I kept only the first representation of the duplicate as it could potentially create a bias during training. A bigger issue was that for 347 molecules the HOMO/LUMO dataset contained energy measurements for 245 molecules (HOMO) and 233 molecules (LUMO), with many molecules lacking one or both orbital energy values. HOMO energies ranged from -6.13 to -4.86 eV with a mean of -5.18 eV and standard deviation of 0.19 eV. LUMO energies spanned -3.79 to -1.06 eV with a mean of -2.93 eV and standard deviation of 0.34 eV.

Later, I added another 232 molecules from the SQL database using DBeaver, this extension contained no duplicates. And also, already contained some calculated features such as number of bonds for all types of bonds going from single bonds to aromatic. These features had to be calculated for the HOPV dataset as well before merging.

After I had all data, I decided to merge the full training dataset (HOPV + SQL data) and kaggle submission dataset together to find some similarities and perform analysis, I created number of plots to understand data distribution, I found that some descriptors behave as a classes rather than continuous values, I also found that Ipc feature is 0 for all molecules but 1, where it is astronomically big number, I knew about this problem before as I already tried couple models and I ran into some troubles with this descriptor, I solved it by clipping the values but I could’ve dropped the feature completely.





From there I performed a correlation analysis to find features that correlates the most with homo and lumo values + I found 77 strong correlation pairs between descriptors where their correlation exceeded 0.99 so I can consider their removal.

Data Preparation

In this section, you should document the steps you took to clean, transform, and prepare the data for modelling. This may include removing duplicates, handling missing values, creating new variables and/or merging data sources that will be useful for the final modelling.

Data preparation for this project involved multiple sophisticated steps, transforming raw chemical identifiers into a rich feature space suitable for machine learning. The first critical step was to merge the stability and HOMO/LUMO datasets. After parsing HOPV_homolumo.data to extract experimental values, I performed an inner join on the SMILES strings which served as an identifier. This merge operation connected stability classifications with orbital energy measurements for molecules present in both datasets. That means that out of the original number of 350 molecules, 347 remained after removing duplicates and performing the inner merge, representing molecules with at least partial characterization. The successful merge created a combined dataset with SMILES, stability, HOMO, and LUMO columns, though many rows contained NaN values in the HOMO and LUMO positions due to incomplete coverage.

When this was done and I had a, for now, complete dataset in my hands, I went right into calculating new features to represent smiles in numerical form to make it usable for machine learnings algorithms. I started by calculating RDKit descriptors which converted each SMILES string into 217 molecular descriptors showing diverse chemical properties. This descriptor calculation process later revealed a data quality issue: certain RDKit descriptors produced extreme or invalid values for specific molecules. Most notably, the Ipc descriptor generated values exceeding 10^{25}, I solved this issue by clipping these extreme values to 10^{6}, preventing these outliers from misleading certain models. Later, the SQL dataset contained some bonds/atom counts which I had to calculate for the HOPV dataset as well before merging them together in DBeaver. I did it by extracting only smiles column from HOPV_df and calculating all mentioned bond/atom features (total_bonds, n_single_bonds,n_doubble_bonds, etc.) and adding homo,lumo and stability columns. I divided this freshly created dataset into smaller tables which I imported to DBeaver and merged with the SQL dataset by moving the SQL tables and identical HOPV tables (identical form, not data) into 'Public_data'. There was an issue in this step, the indexing for my SQL private tables and HOPV tables both started at 1, which means that had I moved it as it was, I would have placed both molecular information on the same given index, so I had to merge it one by one while keeping track of the right indexes. After all the tables were merged together, I simply joined them into one and exported them as an csv file which I read as a pd.DataFrame in python and re-calculated all RDKit descriptors. After I had this now full datset in python, I decided to manually calculate some additional features, or probably I should rather say, feature interactions, because it was just a division or multiplication of some already calculated features, each with chemical justification. Besides what I've found on the internet, I even had a meeting with my grandmother (high school chemistry teacher) and my uncle (double master's degree in medicine) about whether there are some features that are possible to calculate from the smiles string itself, and have some effect on stability or homo/lumo energy level. After my own efforts and their advice, I ended up with about 40 extra feature interactions. With features computed, we implemented a rigorous preprocessing pipeline to ensure data quality.

First was Missing Value Handling - All NaN values in descriptors were filled with zeros. While filling it with mean value might seem as a better approach for some of you, It could create a bias as all features were strongly dependent on the smile itself, and some NaN values represented a complete absence of the given structural feature for the given molecule rather than error in computation. Next was Percentile Clipping – I clipped each feature to its 1st and 99th percentiles to remove outliers while preserving the overall distribution shape. This mild clipping removed extreme outliers without overly constraining the data range. Lastly, constant feature removal – feature showing zero variance across the dataset were removed as they provided no added value to ML algorithms, only increased dimensionality.

For regression, I also implemented clustering to determine sample-weights by combining the training data (466 HOMO, 454 LUMO samples) with the Kaggle test molecules (200 samples) and performing K-means clustering in the scaled feature space. Using the elbow method to identify optimal cluster counts. For each cluster, I counted how many test molecules it contained, then computed sample weights for training molecules based on two factors: the number of test molecules in their cluster (indicating relevance to the test distribution) and their distance to the cluster center (indicating how ''reliable'' they are within the cluster) this helped the model to focus on patter that are more relevant for test predictions.

After data was cleaned, we used stratified 80/20 train-test splits to ensure balanced representation of target values in both training and validation sets for both classification and regression models' training.

Classification

For stability classification, I selected Logistic Regression with L1 (Lasso) regularization as our final model after a long experimentation with multiple algorithms. I made this choice after a systematic comparison of Random Forest, Gradient Boosting, and various linear models, where Logistic regression performed the best. This was probably because despite the apparent complexity of molecular stability, my analysis showed that the decision boundaries were mostly linear. My theory is that this was caused by the ‘’curse of dimensionality paradox’’ where in a high dimensional space it might happen that even complex patterns become linearly separable. Also, the L1 regularization filtered features from 240 to around 80-100 preventing overfitting due to the limited sample size. With this done, I went to hyperparameter tuning. I employed Optuna, to systematically search for the hyperparameter space. After 100 trials with 5-fold cross-validation, Optuna identified optimal parameters: C=0.409 (strong regularization), penalty='l1', indicating that aggressive feature selection improved generalization. The relatively small C value confirmed our concern about overfitting and so that without strong regularization, the model would memorize noise in the training data rather than learning generalizable patterns. Slowly heading towards the end of my efforts to improve the model, the biggest breakthrough (from 0.8 to 0.85 F1 score) I achieved by adding new 35 chemistry-informed features such as conjugation features, heteroatom features, charge distribution, etc. I did this after a long consultation with experts in the field.

To address the slight class imbalance I used SMOTE (only to training set). SMOTE generates synthetic minority class examples, creating a balanced training set. We applied SMOTE after the train/test split of course, applying it before would have caused a data leakage. I also used label encoder to encode stability columns into numerical values, and StandartScaler() to scale the features so they have similar range of possible values, of course, the scaling was done also after train test split.

After optimizing the base Logistic Regression model, I explored ensemble methods to potentially improve performance further. We tested Voting Classifiers (combining Logistic Regression and Random Forest with soft voting) and Stacking Classifiers (using multiple base models with a meta-learner). Surprisingly, stacking actually performed a little worse, and voting classifier provided no improvement over the single Logistic Regression model; it performed the same. Later I did a quick analysis and found out that both models included in voting classifier (Logarithmic Regression and Random Forest) were in 98% agreement (only one feature was predicted differently).

To explore the true generalization capability, we ran a proper 5-fold cross-validation with SMOTE applied within each fold (not to the entire dataset). The cross-validation showed a mean F1-score of 0.8831 +- 0.0190, indicating robust and stable performance. The low standard deviation (0.019) confirmed that the model generalizes well across different data partitions rather than memorizing patterns specific for training only.

Analysis of L1-regularized coefficient magnitudes revealed which molecular characteristics most strongly predict stability. The features with largest absolute coefficients included conjugation-related descriptors (pi_electrons, conjugation_score, aromatic_ratio), heteroatom density measures (nitrogen_density, hetero_ratio), and charge distribution characteristics (charge_range, MaxPartialCharge). This pattern aligns with established chemistry.

To wrap this up, the final classification model achieved an F1-score of 0.85 on the Kaggle public leaderboard, representing strong predictive capability given the limited training data.

Regression

I decided to use two SVR models for homo and lumo predictions, one for each. I always planned to use two separate models as they both, homo and lumo, have very different dependencies and patterns to be learned. I experimented with all kinds of models from linear through forests to more sophisticated models like LGBMRegressor, after a long experimentation, SVR options outperformed the others.

For HOMO energy prediction, our exploration of algorithms revealed that Support Vector Regression (SVR) with a radial basis function (RBF) kernel significantly outperformed traditional linear methods. While my initial experiments with Ridge regression achieved reasonable performance (MAE ≈ 0.12 eV), SVR's non- linear approach exceeded this performance. Critical to SVR's success was proper feature scaling. Unlike tree-based methods that are scale-invariant, SVR's distance-based and so it requires standardized features. We applied StandardScaler transformation, ensuring all features had zero mean and unit variance before model training. Another effective trick for homo predictions was already mentioned clustering- based sample weighting, DVR's learning capacity is determined by it's number of support vectors, clustering helped to focus this learning capacity on the most predictions-relevant patterns. In effort to push performance even further I tried an ensemble method, I combined 3 SVR models with different configurations, an RBF kernel model (optimized via Optuna), a polynomial kernel degree-2 model, and a polynomial kernel degree-3 model. Each kernel captures different types of non-linearity. RBF for smooth radial patterns, low-degree polynomials for gentle curvature, higher-degree polynomials for sharper transitions. The final HOMO prediction averaged these three models with weights (60% RBF, 20% poly-2, 20% poly-3), determined by their respective cross-validation performance. However, although I did not check, they must have been in a strong agreement because it increased performance only very slightly.

For LUMO prediction, I employed the same SVR framework as HOMO but with different hyperparameter configurations which were also fine-tuned using Optuna with 250 trails. For the LUMO ensemble, we combined two SVR variants: an RBF kernel model (the one optimized by Optuna) and a polynomial degree-2 model. I omitted higher-degree polynomials for LUMO as they showed signs of overfitting during cross-validation. The final LUMO prediction averaged these two models with weights (70% RBF, 30% poly-2). The sample weighting strategy proved equally valuable for LUMO prediction. Training molecules structurally similar to the test distribution (identified through clustering) received elevated weights, which made the patterns learned by the model more usable for the use-case molecule selection.

At the end, when the kaggle competition was over (unfortunately) and I was trying to re-construct my model pipelines to fit the task form requirements, I decided to not use percentile clipping after all. When I tried to submit it on kaggle to see whether it did not hurt the model, as a surprise, it improved the performance from 0,08 MAE to 0,072. Unfortunately, it was after the deadline, otherwise it would have ranked me 6th , but I can proudly say that my model performs better than I expected.

While Support Vector Regression does not provide direct feature importance scores like tree-based methods or linear model coefficients, our previous analysis showed that for HOMO prediction, nitrogen-related features (nitrogen_density, nitrogen_aromatic, count_n) consistently showed strong associations with higher (less negative) HOMO energies. Conjugation measures (pi_electrons, conjugation_score, aromatic_ratio) also demonstrated high importance. Charge distribution descriptors (charge_range, MaxPartialCharge) captured the influence of molecular polarity on orbital energies. For LUMO prediction, electron-withdrawing group indicators (halogen counts, oxygen_density) and polarity measures (TPSA, aromatic_polarity) emerged as influential features. The unsaturation descriptor, quantifying bond multiplicity, proved particularly valuable for LUMO as well. Same as with classification our chemistry-driven features I created after the consultation with my family-experts, although they did not make such a big difference as they did in classification model, they proved to be valuable as they consistently appeared in importance rankings.

6 Clustering

During iteration 2 I performed HDBSCAN clustering on my first 350 molecules, xy_df dataset (without SQL molecules) first I made the mistake to perform it without scaling the features, Before scaling, the model identified a lot of clusters (13), after I applied StandartScaler() to scale the features before fitting HDBSCAN it identified only 4 clusters (cluster (-1) as samples that model could not identify and then 4 remaining clusters). To verify clusters, I ran sophisticated analysis of each cluster's chemical properties as it should have some chemical credibility. Only thing I could find was after analysing their stabilities, cluster 0 and 1 were holders of smiles with 'low' stability only while cluster 3 had only smiles with 'medium' stability, however, cluster 2 had many smiles with 'high' and 'medium' stability, very little with 'low'. This could be a helpful insight that if a sample is classified as cluster 0 or 1 it is very likely to have low stability, when if it classifies as cluster 3, it is likely to have medium stability.

Later, during iteration n I performed KMeans clustering, this time on full dataset with SQL dat. I loaded the training dataset, kaggle dataset, added dataset_source column to both to keep track which molecule is from which dataset and merge them (smiles only). After that I re-calculated all descriptors, replaced NaNs and infinities with 0, removed constant features and scaled the rest using StandartScaler(). From there I determined optimal number of clusters using elbow method which ended up being 3 and performed the clustering itself. The results showed that all kaggle molecules were in one cluster together with 301 other molecules, this gave me an idea for my regression model to apply similar clustering as a part of preprocessing and assign a different weight to each sample based on how similar it is to kaggle, this will allow the model to learn patterns more relevant for kaggle, which improved its performance, but it will cause worse performance when it gets a random molecule that is widely different from those in kaggle dataset (As I could see in PCA, kaggle molecules were pretty similar to each other)

Limitations

In this section, you should identify and discuss the key limitations of your machine learning approach and models. You can address constraints in your data, methodology, model performance, or assumptions, as well as potential impacts these limitations might have on real-world application of your results.

Despite achieving promising results, our machine learning approach faces several important limitations that must be acknowledged for appropriate application of these models in real-world scenarios.

Most likely the biggest limitation comes from the dataset size, all my efforts for feature selection for both classification and regression model ended up lowering the performance, I included strong regularization and careful cross- validation but the high dimensionality-to-sample ratio (240-275) features for 450-466 molecules means the risk of overfitting still remains. Additionally, the SMOTE synthetic oversampling (used for classification) creates artificial molecules. While these synthetic examples help balance class distributions, they may not represent chemically valid structures. The algorithm learns from these molecules as well, which could introduce bias if the interpolated structures don't reflect real chemical space. And lastly, my final models prioritize interpretability, but this comes at a potential performance cost. More sophisticated approaches like neural networks, graph neural networks operating directly on molecular graphs, or deep learning models trained on molecular fingerprints might achieve better predictive accuracy. However, such black-box models would sacrifice the insights we extract from feature importances and model coefficients.

Conclusion

This project successfully developed machine learning models for predicting stability classification and HOMO/LUMO energies of organic photovoltaic materials, achieving meaningful accuracy levels despite various challenges. For the classification task, I optimized Logistic Regression model with L1 regularization achieved an F1-score of 0.85, For regression, separate specialized models for HOMO (SVR 3 models ensemble) and LUMO (2 SVR models ensemble) achieved combined Kaggle MAE of approximately 0.078

For classification I would say the biggest impact was the feature engineering and domain knowledge (not mine but my relatives – experts) closer to the end (f1 score = 0.78 and higher), sophisticated models and model adjustments delivered none, or very minor improvements in model's performance, but calculating a well chemistry-based features significantly enhanced the performance right away. On the other hand, I learned that I have to be careful with the dimensionality of the training dataset, adding the very same chemistry-based features to my regression model to cause overfitting and hurt its performance. Too few features and model miss important patterns, too many features and model overfit to noise. From a business standpoint, my models could evaluate thousands of candidates and pick top 5-10% in hours, assuming 85% f1 score and 0.078 MAE. This would reduce the time and experimental costs drastically.

For next steps, my very first step would be to get more data, as much as possible, also adding graph based molecular representation and using graph neural network could higher the performance but would've lost the insight we got. Also implementing a sort of loop where models most uncertain predictions are manually experimentally measured and then given back to the model's training set.

To evaluate the whole project, while my models cannot replace the experimental measurements, they can be useful tools to accelerate the discovery process and in this use case, rapidly narrows the selection of all possible molecules down to 5-10% selecting the best candidates.

References

Classification model pipeline: here
Regression model pipeline: here
Clustering: here



Games



Leisure & Events



Tourism



Media



Data Science & AI



Hotel



Logistics



Built Environment



Facility

Mgr. Hopmansstraat 2
4817 JS Breda

P.O. Box 3917
4800 DX Breda
The Netherlands

PHONE
+31 76 533 22 03
E-MAIL
communications@buas.nl
WEBSITE
www.BUas.nl

CREATING MEANINGFUL EXPERIENCES

Breda University
OF APPLIED SCIENCES